SRE STUDIUM PRZYPADKU

AZURE ARC HYBRID CLOUD LAB

Tygodnie 1-3: Kubernetes, Azure Arc, Terraform, CI/CD, Ingress TLS, HPA, NetworkPolicy, RBAC

Autor: Bartosz Suszko

Firma: IT Solutions Bartosz Suszko, Olsztyn

Data: Maj 2026 (Tygodnie 1-3)

GitHub: github.com/buth11/sre-homelab-azure-arc

Stack: Kubernetes | Azure Arc | Terraform | KQL | Azure DevOps | Traefik | HPA | NetworkPolicy | RBAC

Spis tresci

Rozdzial 1: Kontekst i architektura

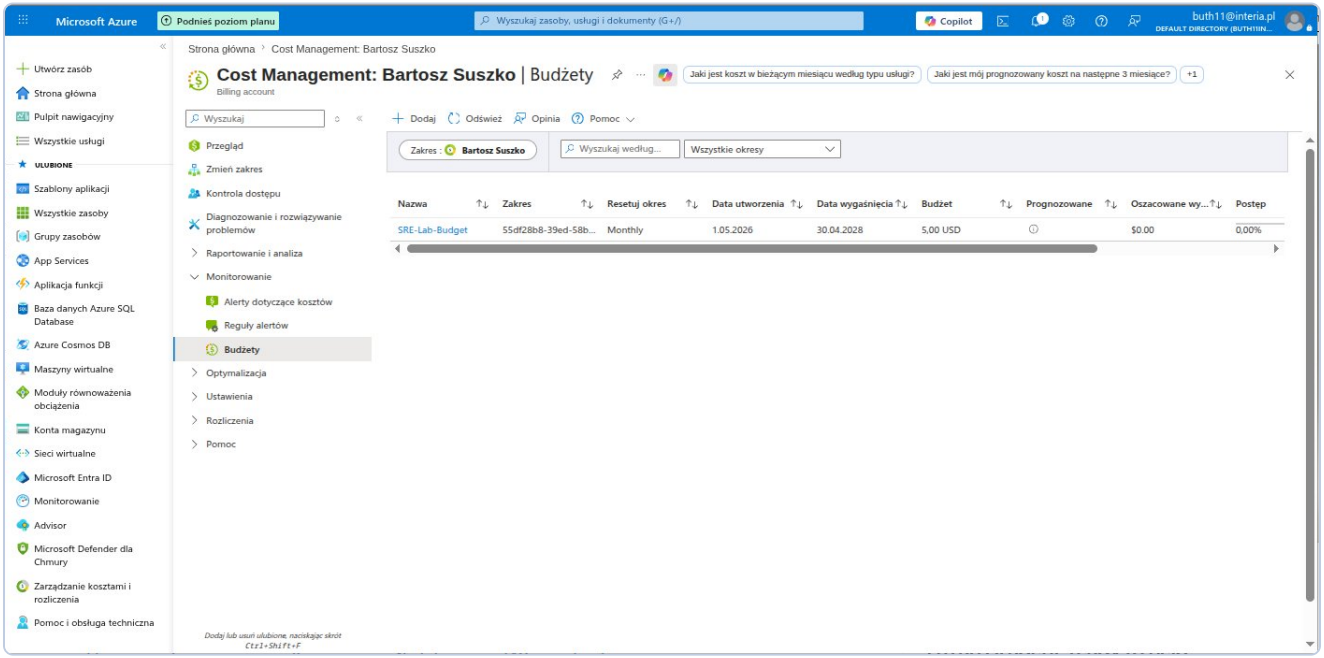
Projekt demonstruje budowe produkcyjnego srodowiska hybrydowego na GMKtec EVO-X2 (128 GB RAM) laczacego Kubernetes z Microsoft Azure przez Azure Arc. Trzy tygodnie praktycznych cwiczen z dokumentacja kazdego kroku i 10 udokumentowanych incydentow.

Warstwa	Technologia	Zastosowanie
Kubernetes	K3s v1.35.4+k3s1	Klaster 3 wezly
Ingress	Traefik + TLS	HTTPS routing po domenie
Autoscaling	HPA (autoscaling/v2)	Skalowanie CPU 2-8 replik
Network Security	NetworkPolicy	Firewall wewnatrz klastra
Access Control	RBAC + ServiceAccount	Least privilege
Cloud Bridge	Azure Arc	Hybrydowa plaszczyzna kontrolna
Monitoring	Azure Monitor + Prometheus	Observability
IaC	Terraform + Remote State	Infrastructure as Code
CI/CD	Azure DevOps	Pipeline z approval gate

Rozdzial 2: Infrastruktura On-Premise

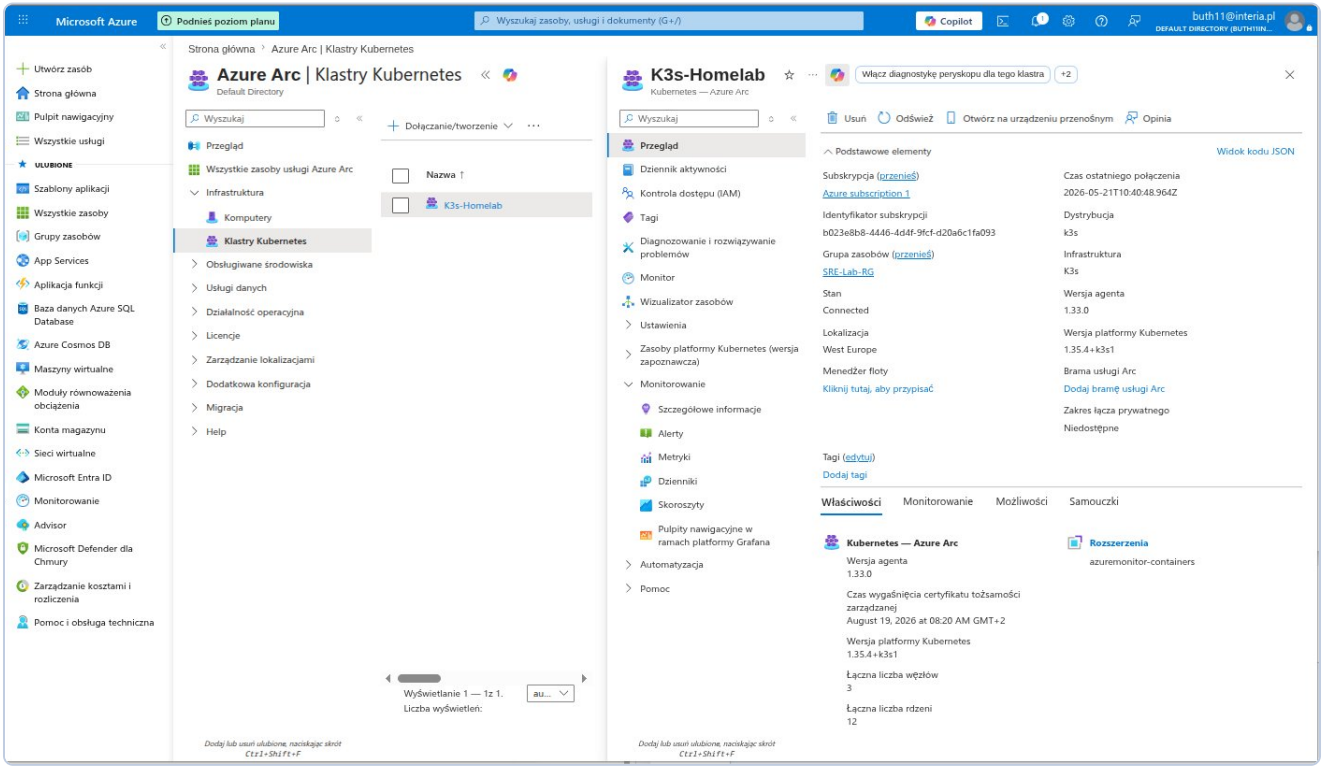
Wezel	IP	CPU	RAM
k3s-master (Control Plane)	192.168.122.10	4 vCPU	8 GB
k3s-worker1	192.168.122.11	4 vCPU	16 GB
k3s-worker2	192.168.122.12	4 vCPU	16 GB

Rozdzial 3: FinOps



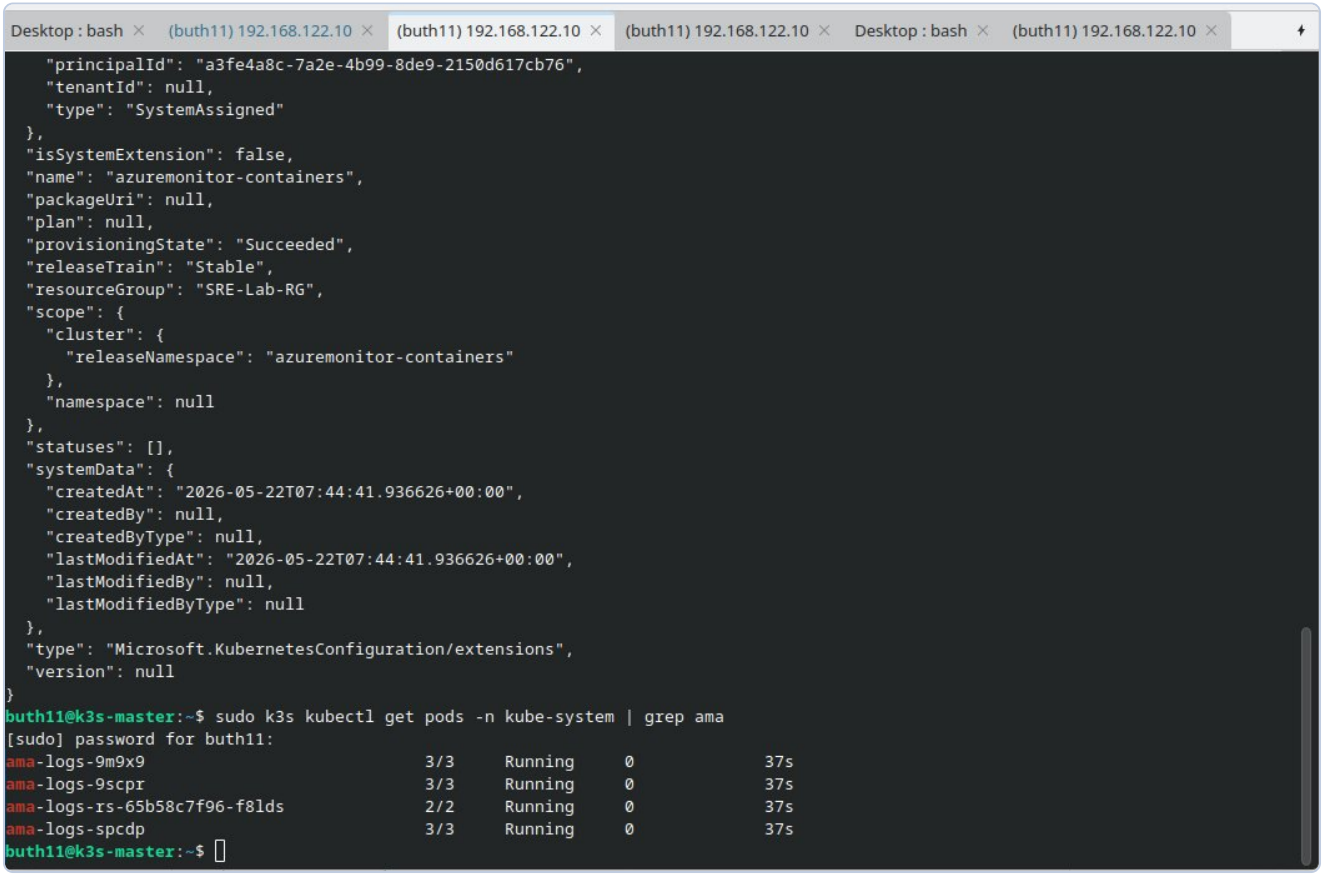
Zrzut 1: Azure Cost Management — budzet 5 USD/mies., koszt: 0.00 USD.

Rozdzial 4: Azure Arc

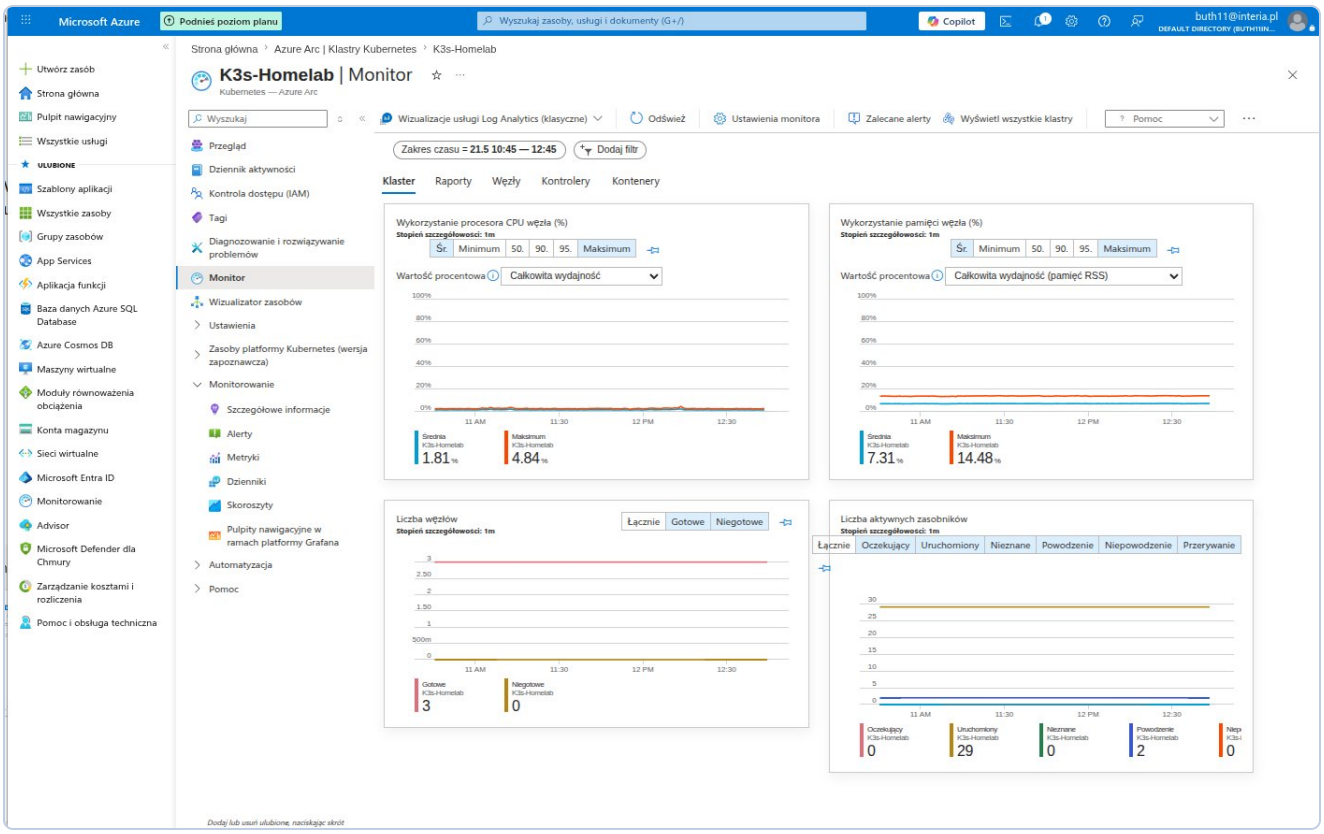


Zrzut 2: Azure Arc — K3s-Homelab Connected, v1.35.4+k3s1, 3 wezly, 12 rdzeni.

Rozdzial 5: Observability

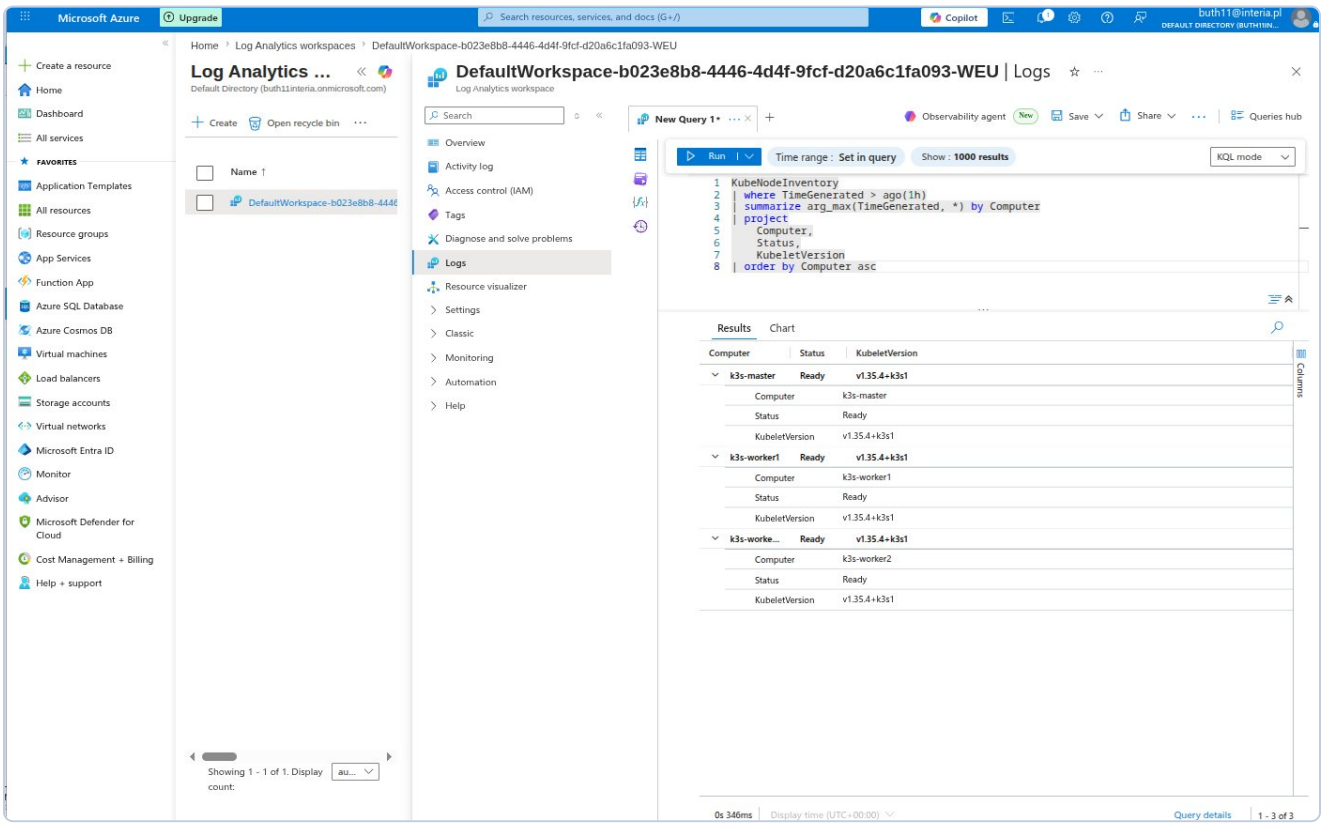


> Zrzut 3: Pody AMA Running — 4 agenty, 0 restartow.

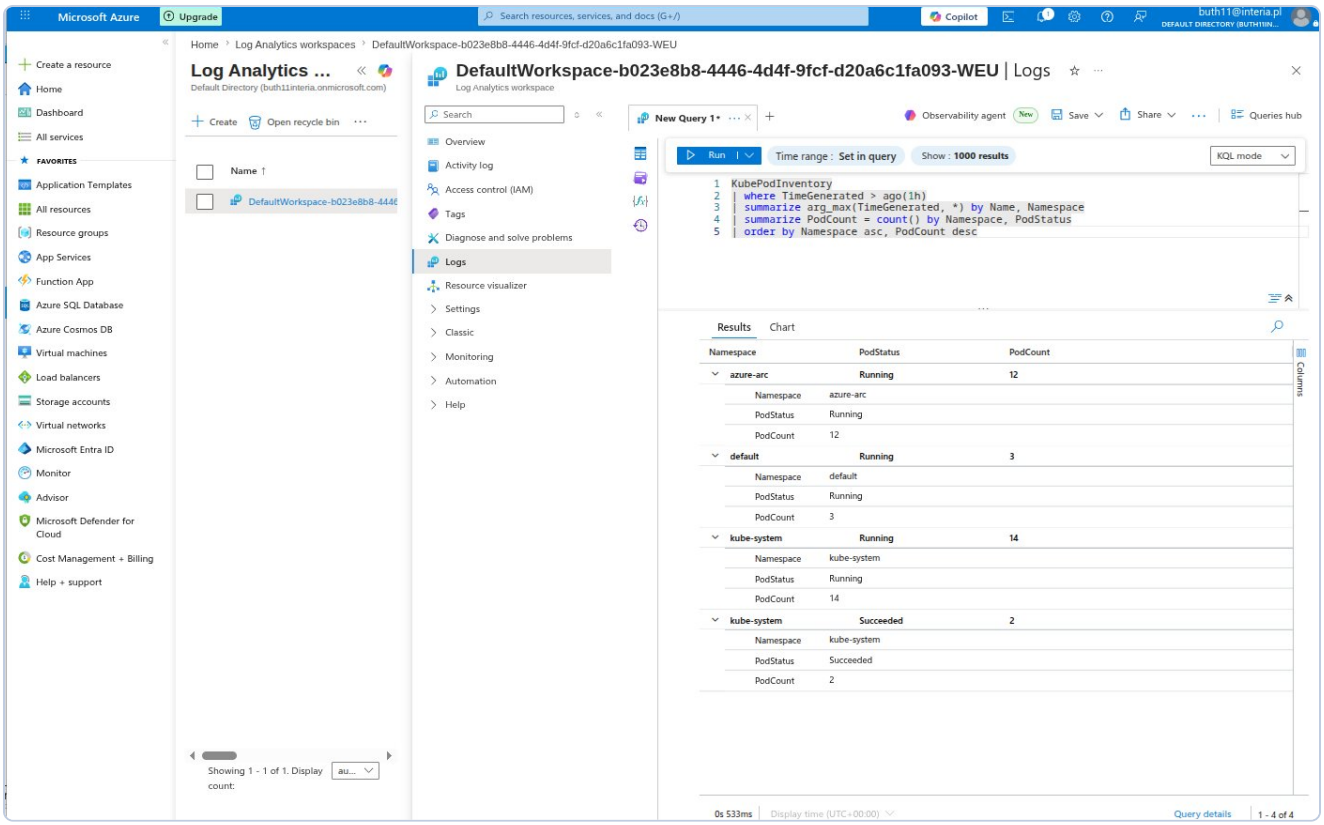


Zrzut 4: Azure Monitor — CPU avg 1.58%, RAM avg 6.27%, 3 wezly Ready, 29 podow.

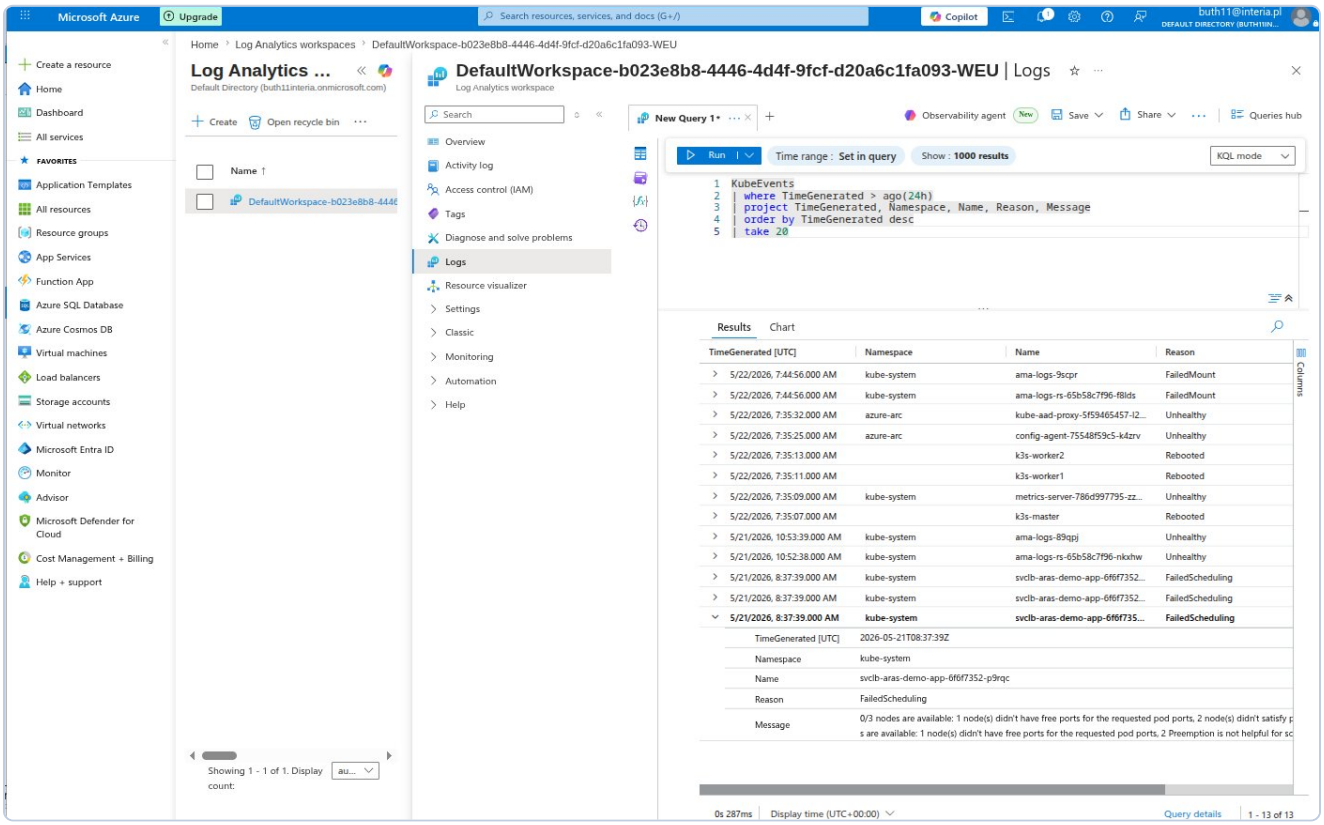
Rozdzial 6: KQL



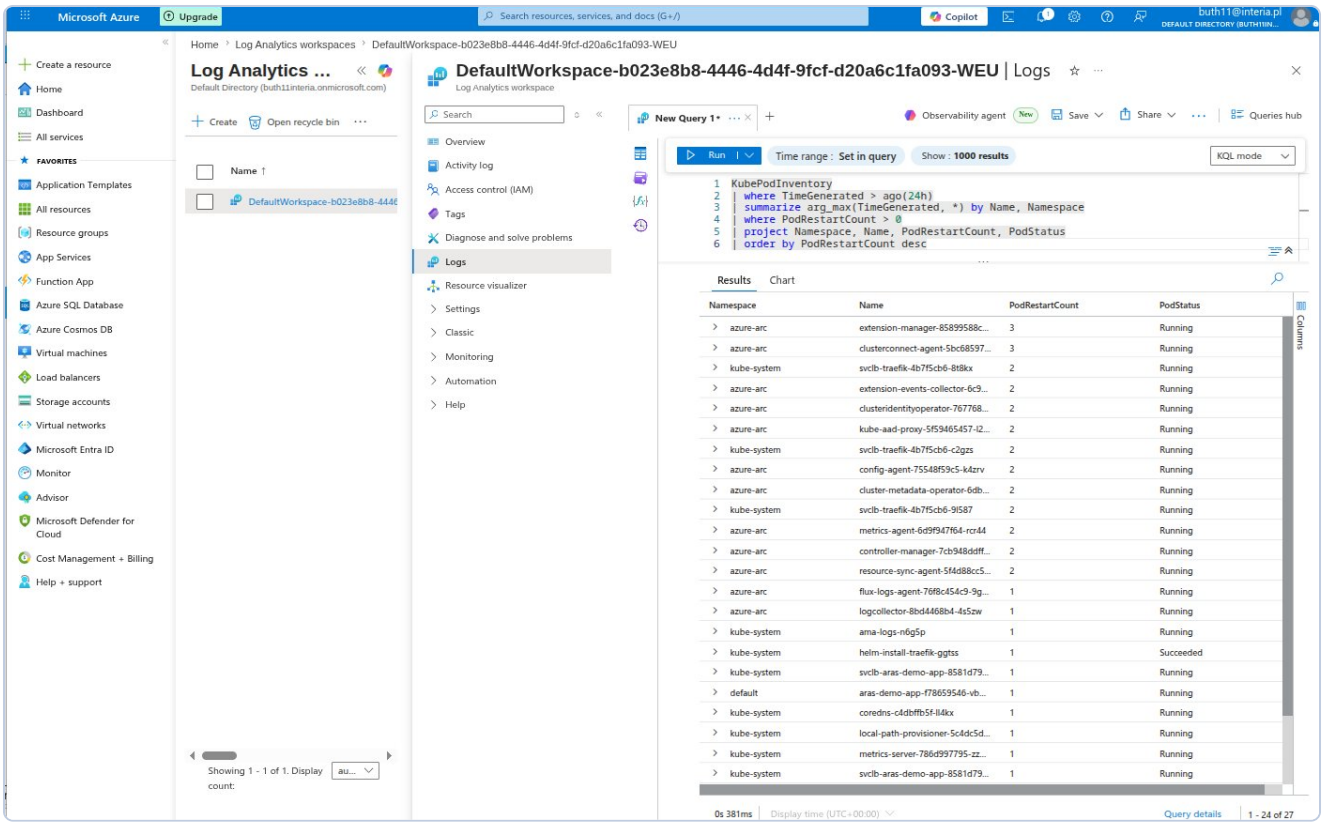
KQL-1: 3 nodes Ready, v1.35.4+k3s1.



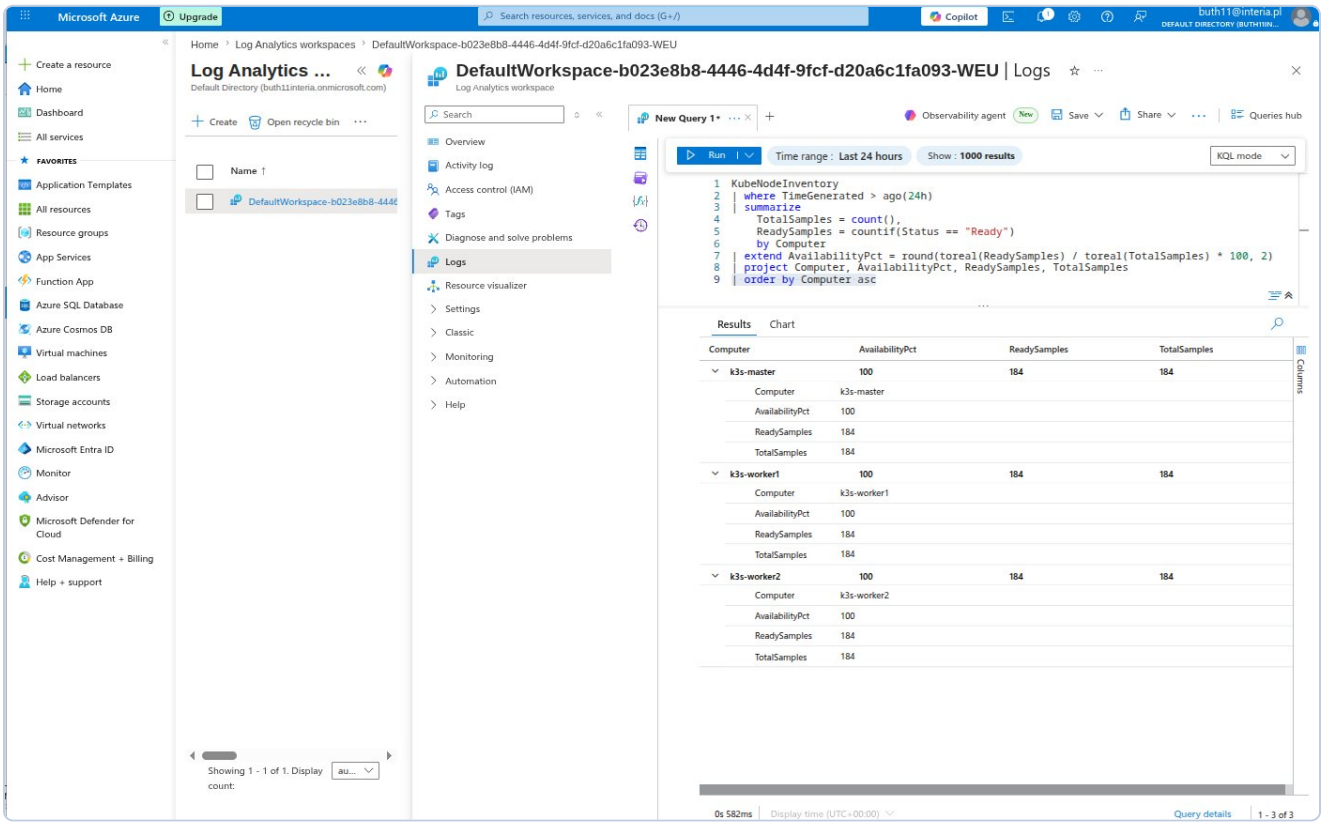
KQL-2: azure-arc 12 Running, default 3 Running, kube-system 14+2.



KQL-3: 13 eventow 24h — Rebooted, Unhealthy (chwilowe), FailedScheduling.



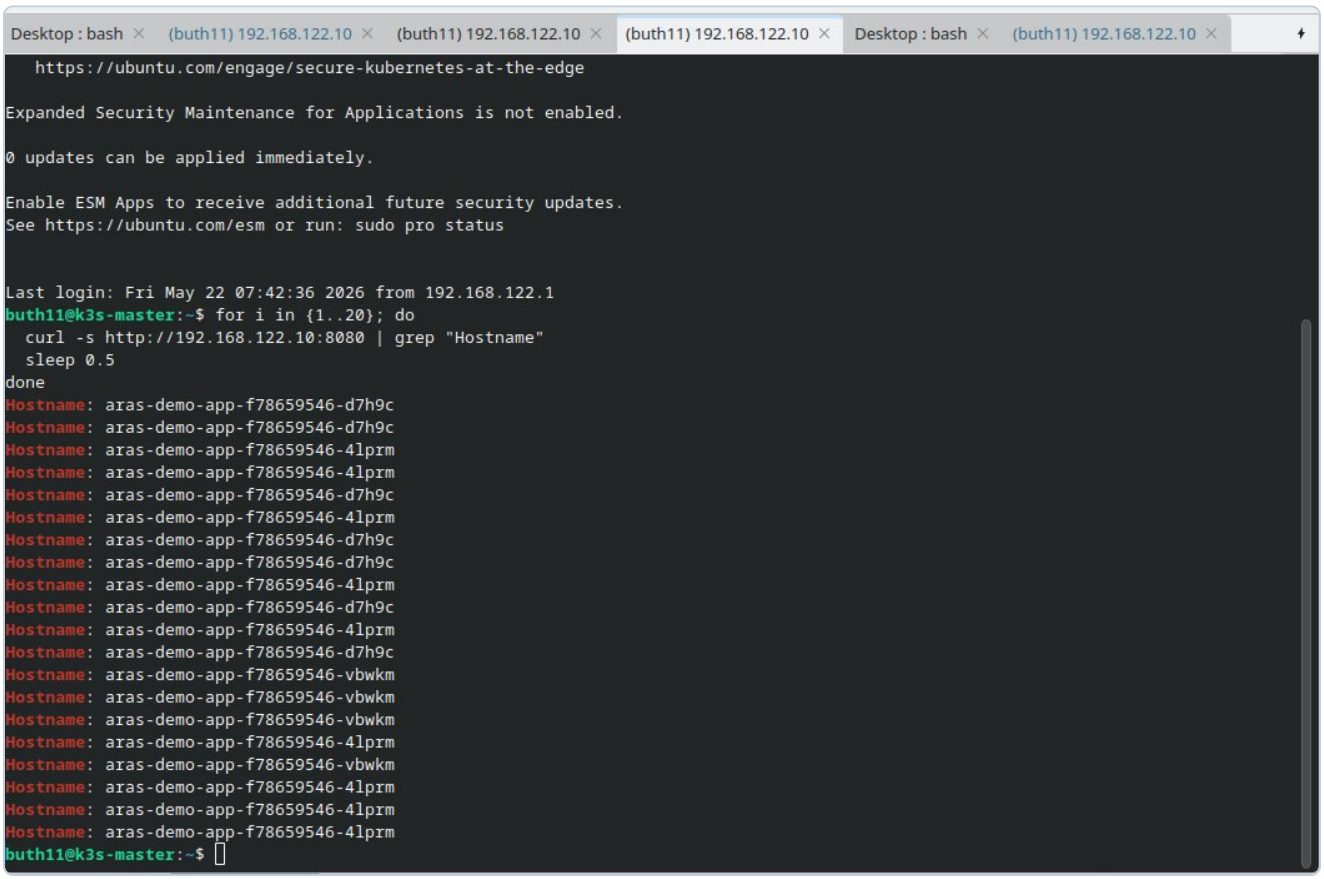
KQL-4: 27 podow z restartami po restarcie VM — wszystkie Running.



KQL-5: SLO 100% — 184/184 probek Ready.

SLO 100% w oknie 24h. Error budget nienaruszony nawet po pelnym restarcie VM.

Rozdzial 7: Load Balancing

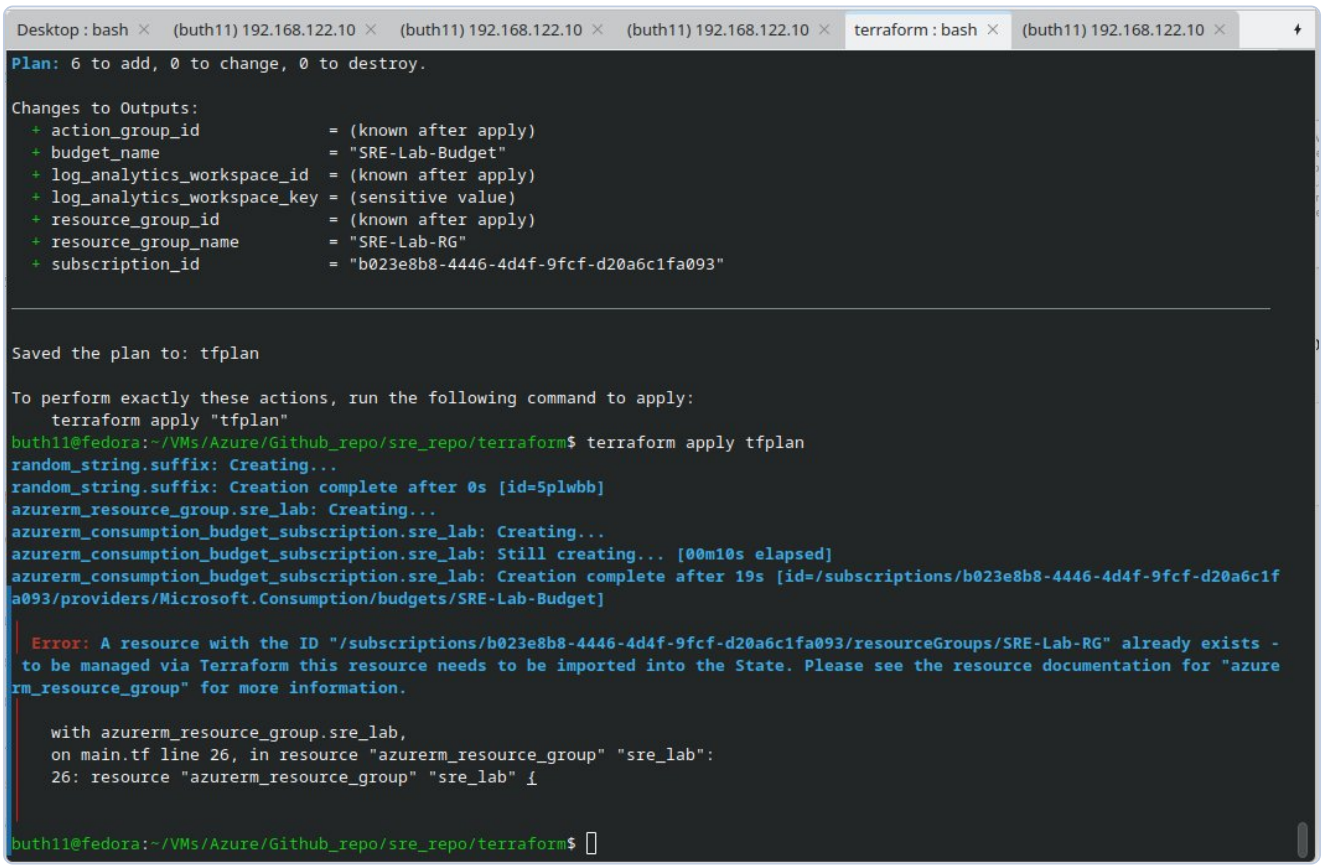


> Zrzut 5: Round-robin — 20 requestow na 3 pody, 900 rownolegla bez bledow.

Rozdzial 8: Prometheus & Grafana

Zrzut 6: Grafana — CPU 2.20%, RAM 12.9%, CPU Quota per namespace.

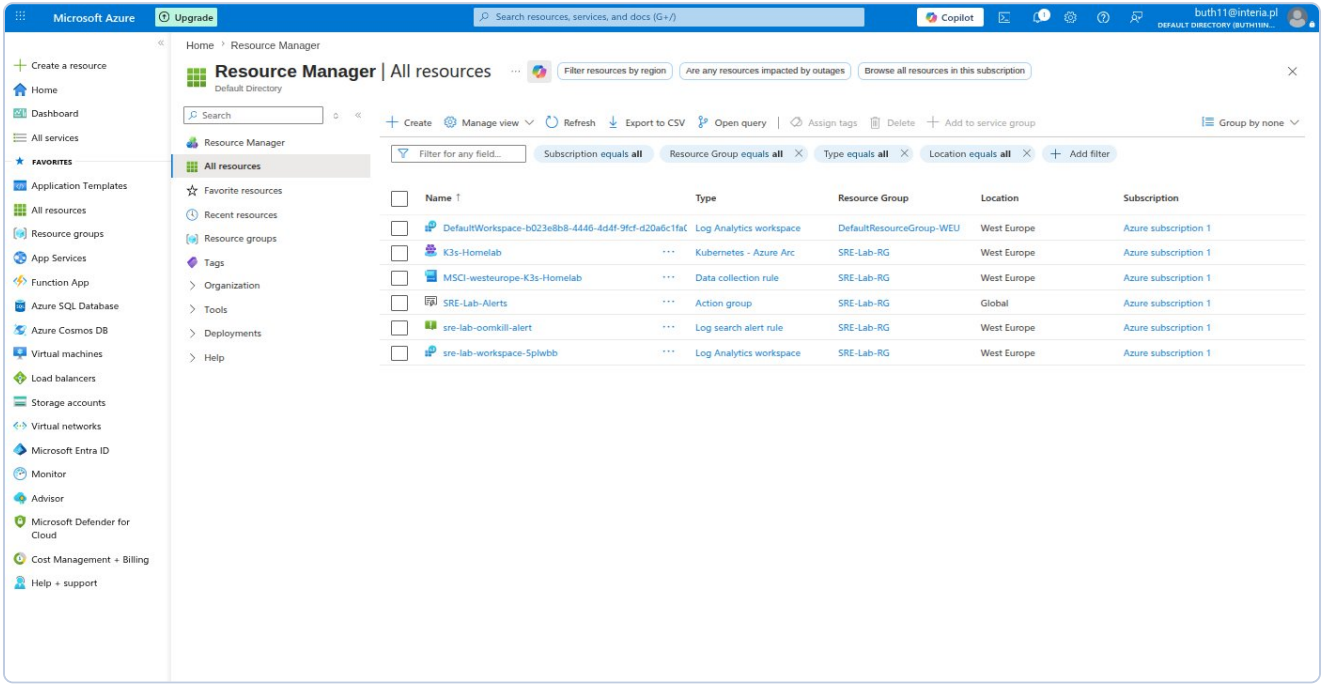
Rozdzial 9: Terraform IaC



> Terraform Plan: 6 zasobow, 0 zmian.

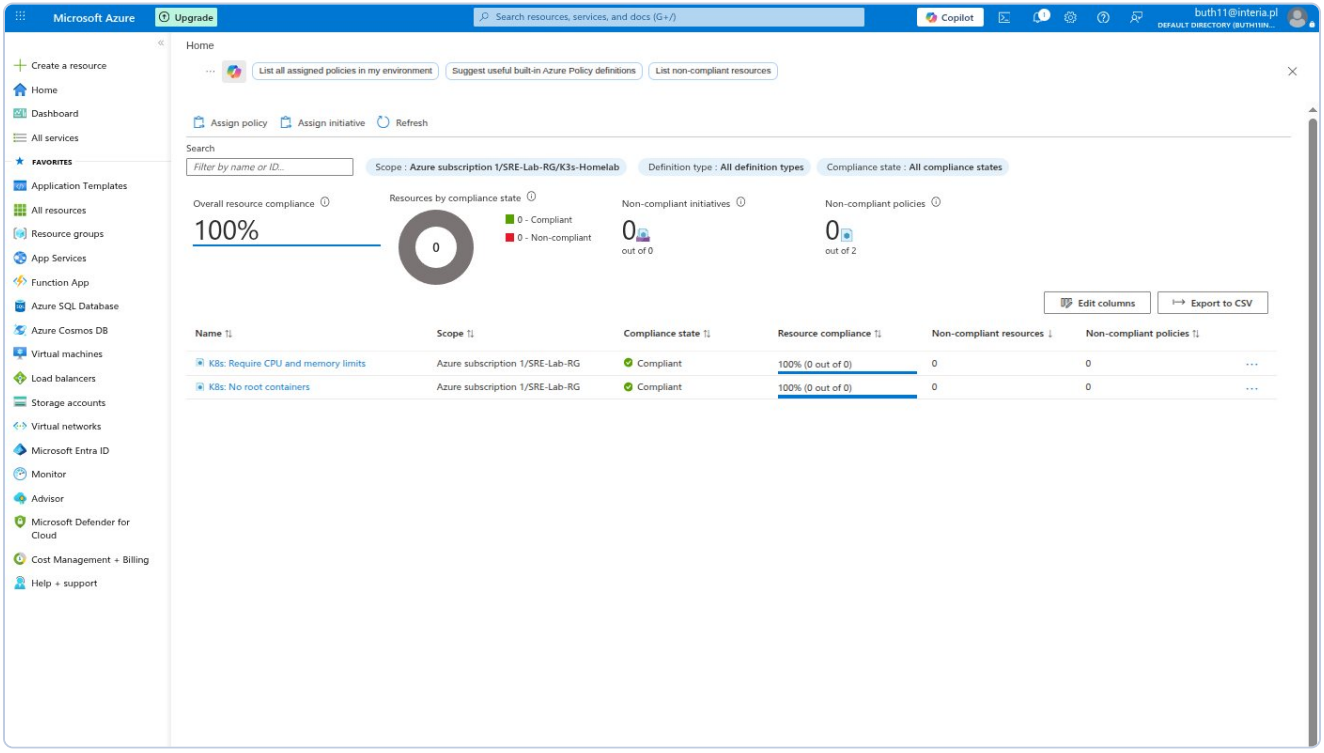
```
Desktop : bash × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × terraform : bash × (buth11) 192.168.122.10 ×

Saved the plan to: tfplan

To perform exactly these actions, run the following command to apply:
    terraform apply "tfplan"
azurerm_resource_group.sre_lab: Modifying... [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG]
azurerm_resource_group.sre_lab: Modifications complete after 3s [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resource
Groups/SRE-Lab-RG]
azurerm_monitor_action_group.sre_alerts: Creating...
azurerm_log_analytics_workspace.sre_lab: Creating...
azurerm_monitor_action_group.sre_alerts: Creation complete after 4s [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/reso
urceGroups/SRE-Lab-RG/providers/Microsoft.Insights/actionGroups/SRE-Lab-Alerts]
azurerm_log_analytics_workspace.sre_lab: Still creating... [00m10s elapsed]
azurerm_log_analytics_workspace.sre_lab: Still creating... [00m20s elapsed]
azurerm_log_analytics_workspace.sre_lab: Still creating... [00m30s elapsed]
azurerm_log_analytics_workspace.sre_lab: Still creating... [00m40s elapsed]
azurerm_log_analytics_workspace.sre_lab: Creation complete after 47s [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/res
ourceGroups/SRE-Lab-RG/providers/Microsoft.OperationalInsights/workspaces/sre-lab-workspace-5plwbb]
azurerm_monitor_scheduled_query_rules_alert_v2.oom_kill: Creating...
azurerm_monitor_scheduled_query_rules_alert_v2.oom_kill: Creation complete after 5s [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d
20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.Insights/scheduledQueryRules/sre-lab-oomkill-alert]

Apply complete! Resources: 3 added, 1 changed, 0 destroyed.

Outputs:

action_group_id = "/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.Insights/ac
tionGroups/SRE-Lab-Alerts"
budget_name = "SRE-Lab-Budget"
log_analytics_workspace_id = "/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.
OperationalInsights/workspaces/sre-lab-workspace-5plwbb"
log_analytics_workspace_key = <sensitive>
resource_group_id = "/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG"
resource_group_name = "SRE-Lab-RG"
subscription_id = "b023e8b8-4446-4d4f-9fcf-d20a6c1fa093"
buth11@fedora:~/VMs/Azure/Github_repo/sre_repo/terraform$
```

Terraform Apply: 3 dodane, 1 zmieniony.



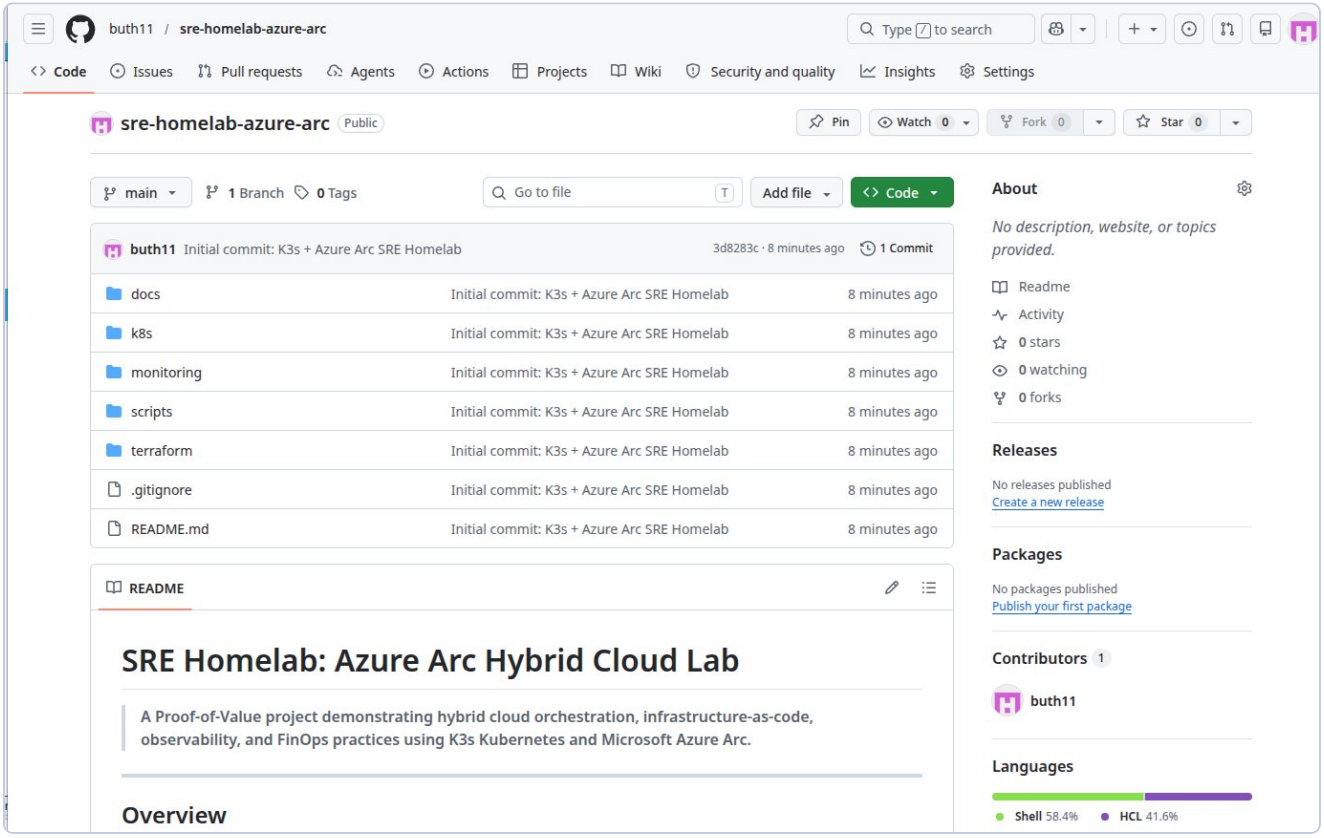
Portal Azure: zasoby Terraform w SRE-Lab-RG.

Rozdzial 10: Azure Policy



Azure Policy: No root containers + CPU/memory limits — oba Compliant.

Rozdzial 11: GitHub



GitHub repo sre-homelab-azure-arc — publiczne, Shell 58.4%, HCL 41.6%.

Rozdzial 12: Czesc 2 — Service Principal, Remote State, CI/CD

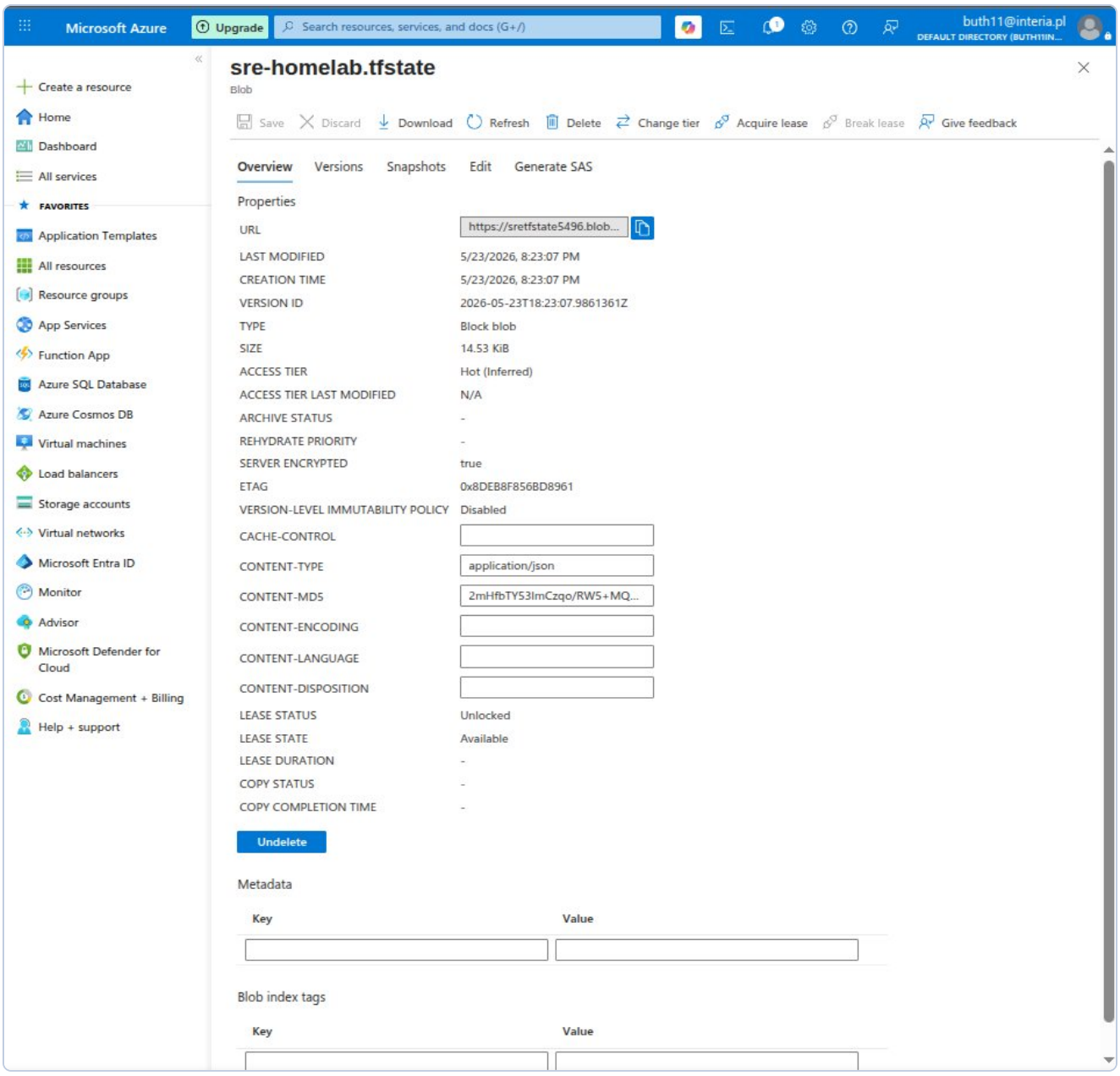
Remote State

Plik sre-homelab.tfstate — 14.53 KiB, szyfrowanie server-side, wersjonowanie aktywne. State lock: 'Acquiring state lock' / 'Releasing state lock' przy kazdym terraform plan.

Pipeline Azure DevOps CI/CD

Variable Group terraform-credentials: ARM_CLIENT_ID, ARM_CLIENT_SECRET (ukryty), ARM_SUBSCRIPTION_ID, ARM_TENANT_ID. Environment production: Approval Gate aktywna.

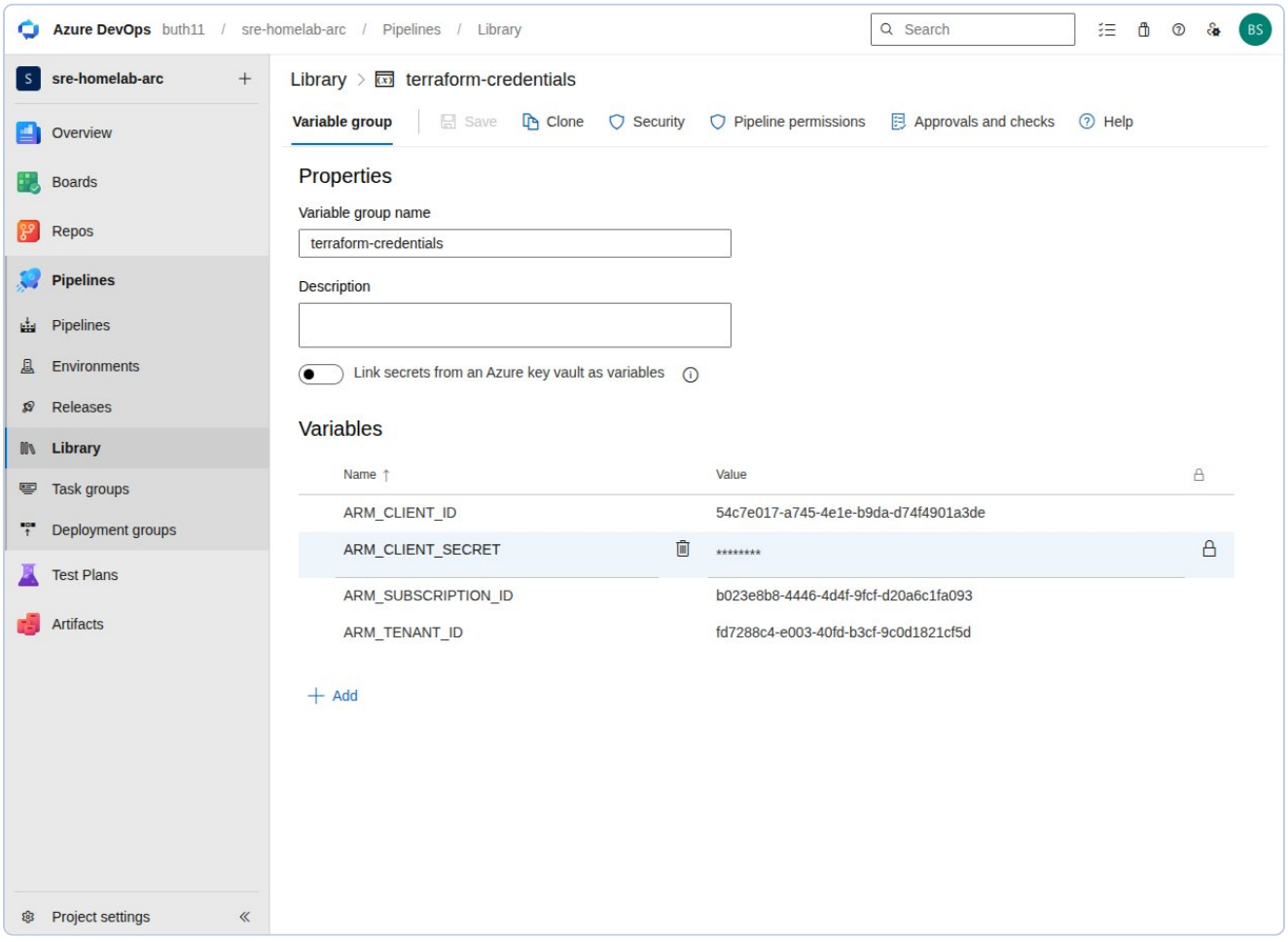
Wynik: Pipeline kompletny — oba stage zielone, 4m 24s. Stage 1 (37s): validate + plan. Stage 2 (49s): apply po zatwierdzeniu.



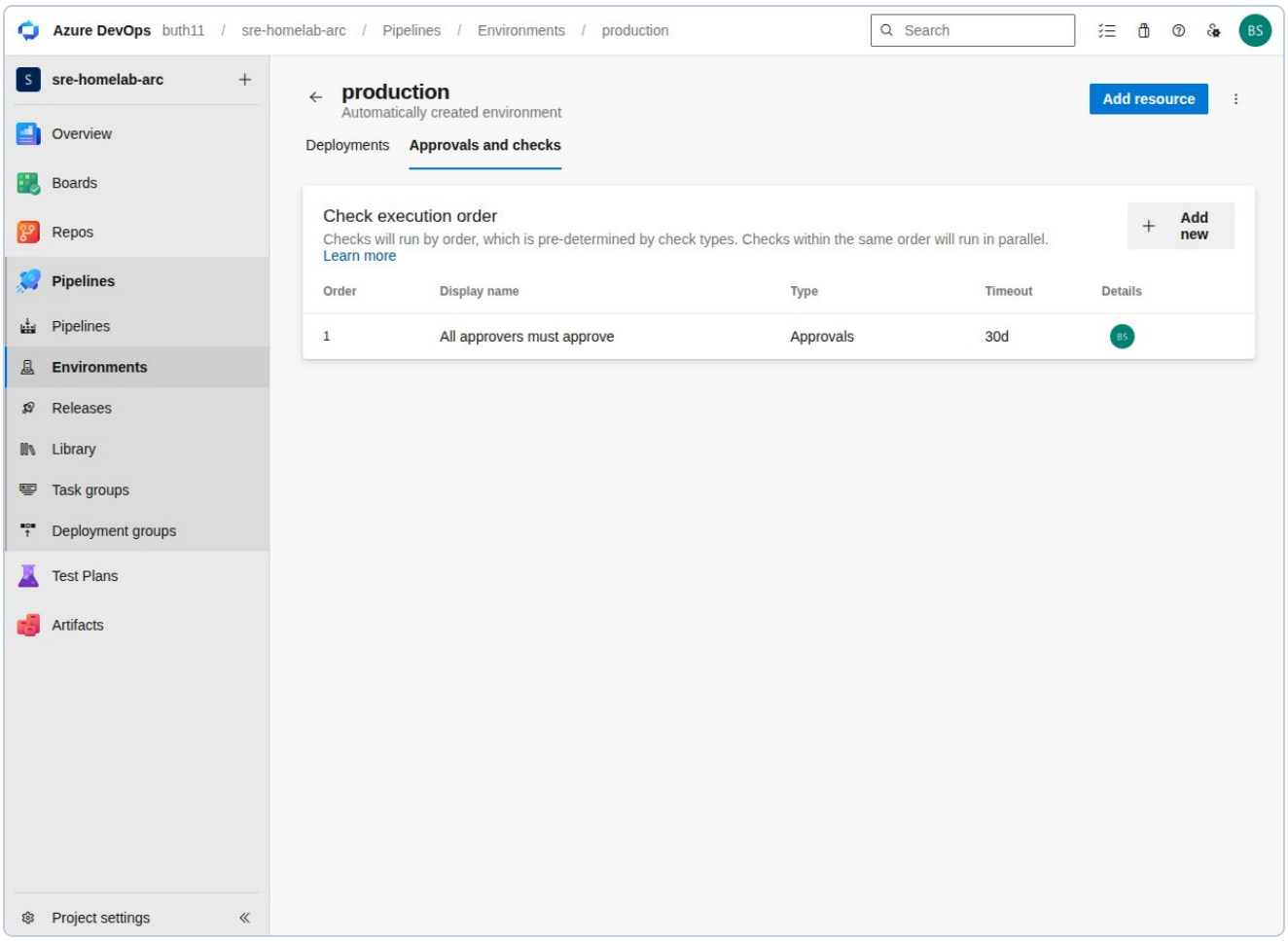
Zrzut W2-2: Portal Azure — blob properties: 14.53 KiB, encrypted, versioning active.

```
buth11@fedora:~/VMs/Azure/Github_repo/sre_repo/terraform$ # terraform plan pobierze state z Azure Blob zamiast lokal
nego pliku
# Powinno pokazac "No changes" bo infrastruktura jest aktualna
terraform plan
Acquiring state lock. This may take a few moments...
random_string.suffix: Refreshing state... [id=5plwbb]
data.azurerm_subscription.current: Reading...
azurerm_resource_group.sre_lab: Refreshing state... [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resource
Groups/SRE-Lab-RG]
azurerm_monitor_action_group.sre_alerts: Refreshing state... [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093
/resourceGroups/SRE-Lab-RG/providers/Microsoft.Insights/actionGroups/SRE-Lab-Alerts]
azurerm_log_analytics_workspace.sre_lab: Refreshing state... [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093
/resourceGroups/SRE-Lab-RG/providers/Microsoft.OperationalInsights/workspaces/sre-lab-workspace-5plwbb]
data.azurerm_subscription.current: Read complete after 1s [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093]
azurerm_consumption_budget_subscription.sre_lab: Refreshing state... [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a
6c1fa093/providers/Microsoft.Consumption/budgets/SRE-Lab-Budget]
azurerm_monitor_scheduled_query_rules_alert_v2.oom_kill: Refreshing state... [id=/subscriptions/b023e8b8-4446-4d4f-9
fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.Insights/scheduledQueryRules/sre-lab-oomkill-alert]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes
are needed.
Releasing state lock. This may take a few moments...
buth11@fedora:~/VMs/Azure/Github_repo/sre_repo/terraform$
```

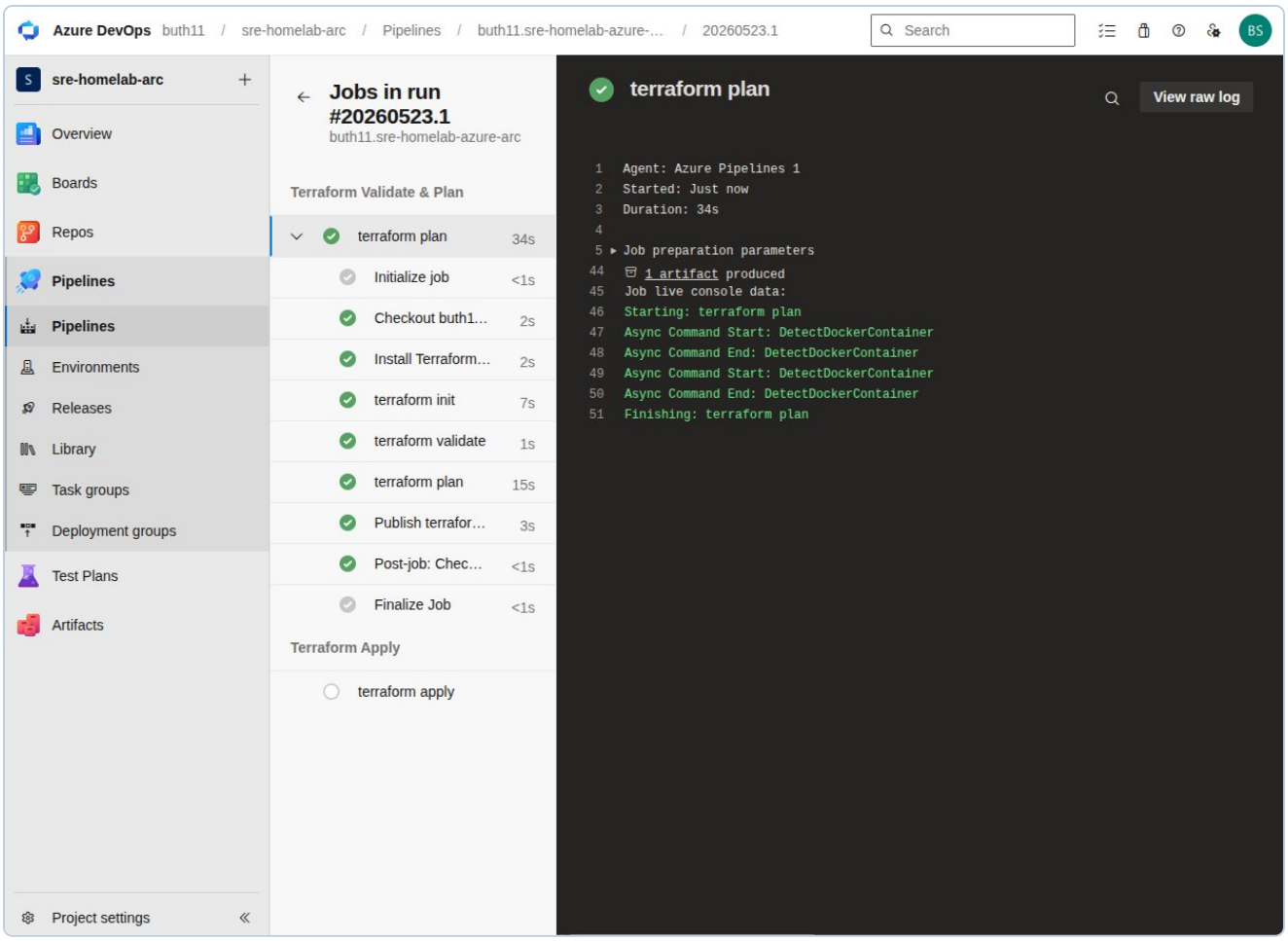
Zrzut W2-3: terraform plan — Acquiring/Releasing state lock. No changes.



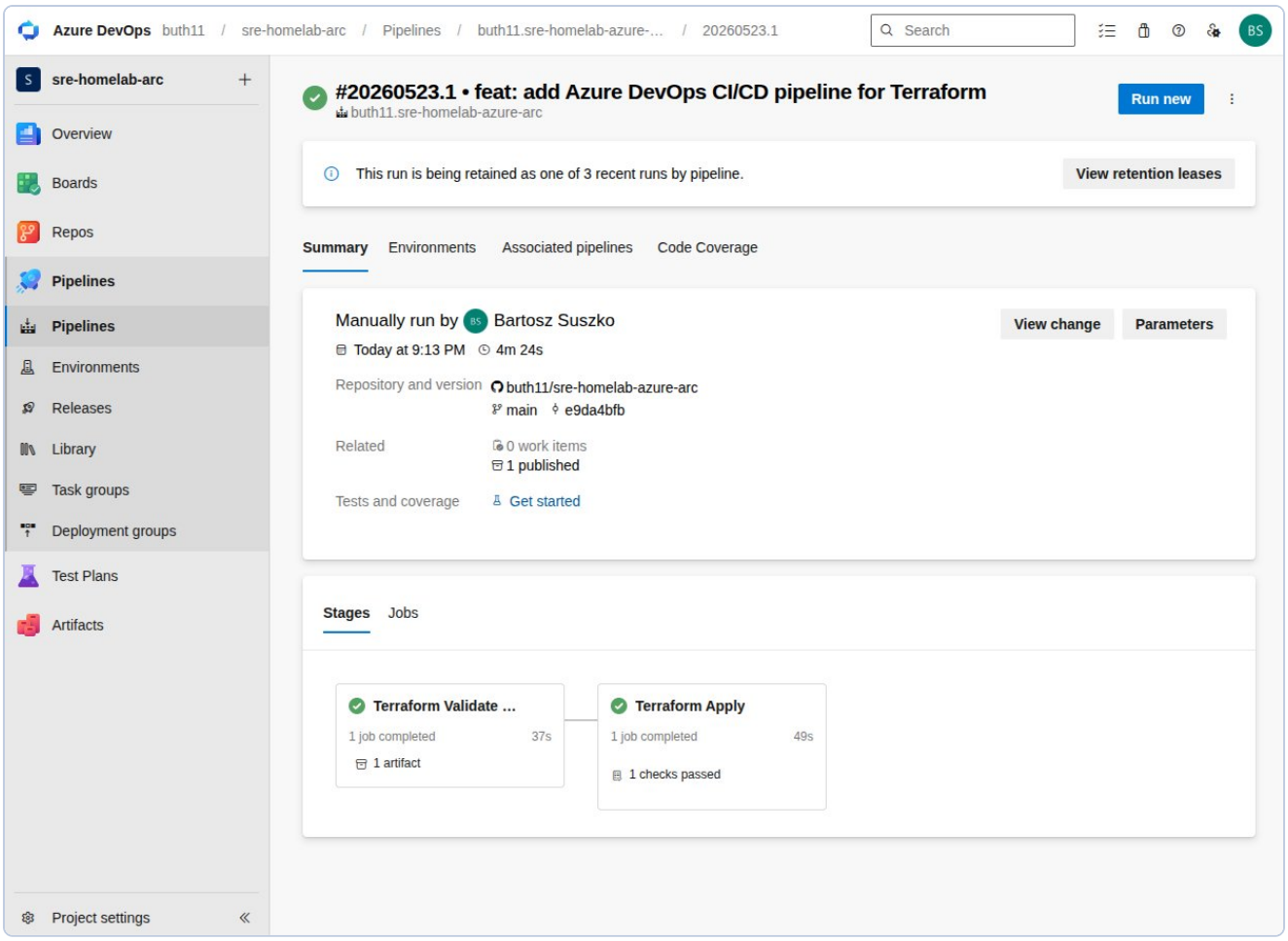
Zrzut W2-4: Azure DevOps Variable Group — ARM_* z ukrytym CLIENT_SECRET.



Zrzut W2-5: Environment production — Approval Gate aktywna.



Zrzut W2-6: Stage 1 — wszystkie kroki zielone, 1 artifact.



Zrzut W2-7: Pipeline kompletny — oba stage zielone, 4m 24s.

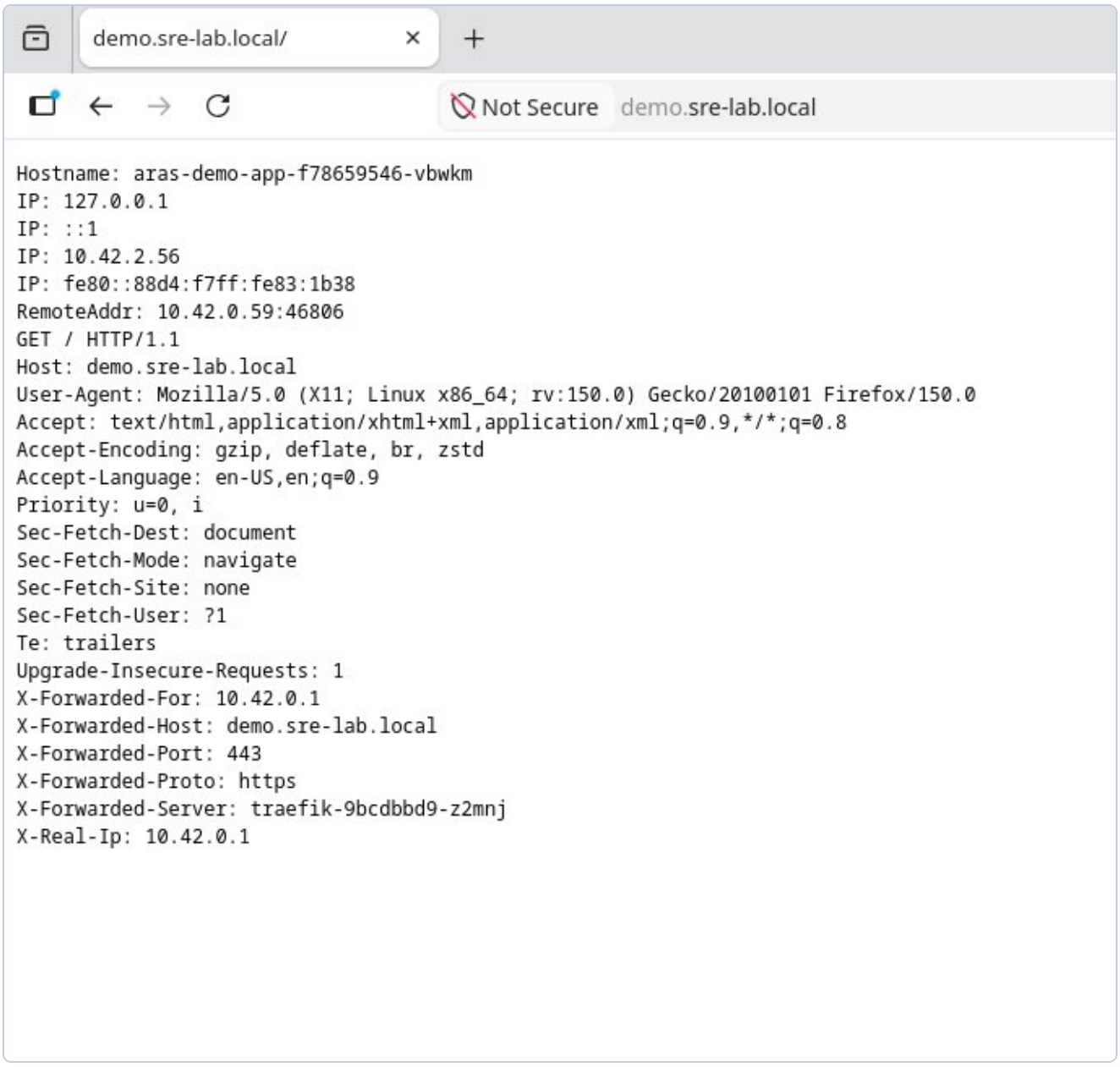
Rozdzial 13: Czesc 3 — Zaawansowane operacje Kubernetes

CZESC 3: Zaawansowane operacje Kubernetes

Ingress z TLS — routing HTTPS po domenie

Traefik Ingress Controller skonfigurowany do HTTPS. Self-signed certyfikat TLS z SAN, zapisany jako Secret Kubernetes. Routing: demo.sre-lab.local → aras-demo-app:8080.

- Certyfikat: openssl x509 self-signed, SAN dla demo.sre-lab.local, wazny 365 dni
- Secret: typ kubernetes.io/tls w namespace default
- Ingress: host-based routing przez Traefik na port 443
- Weryfikacja: X-Forwarded-Proto: https, X-Forwarded-Server: traefik w odpowiedzi

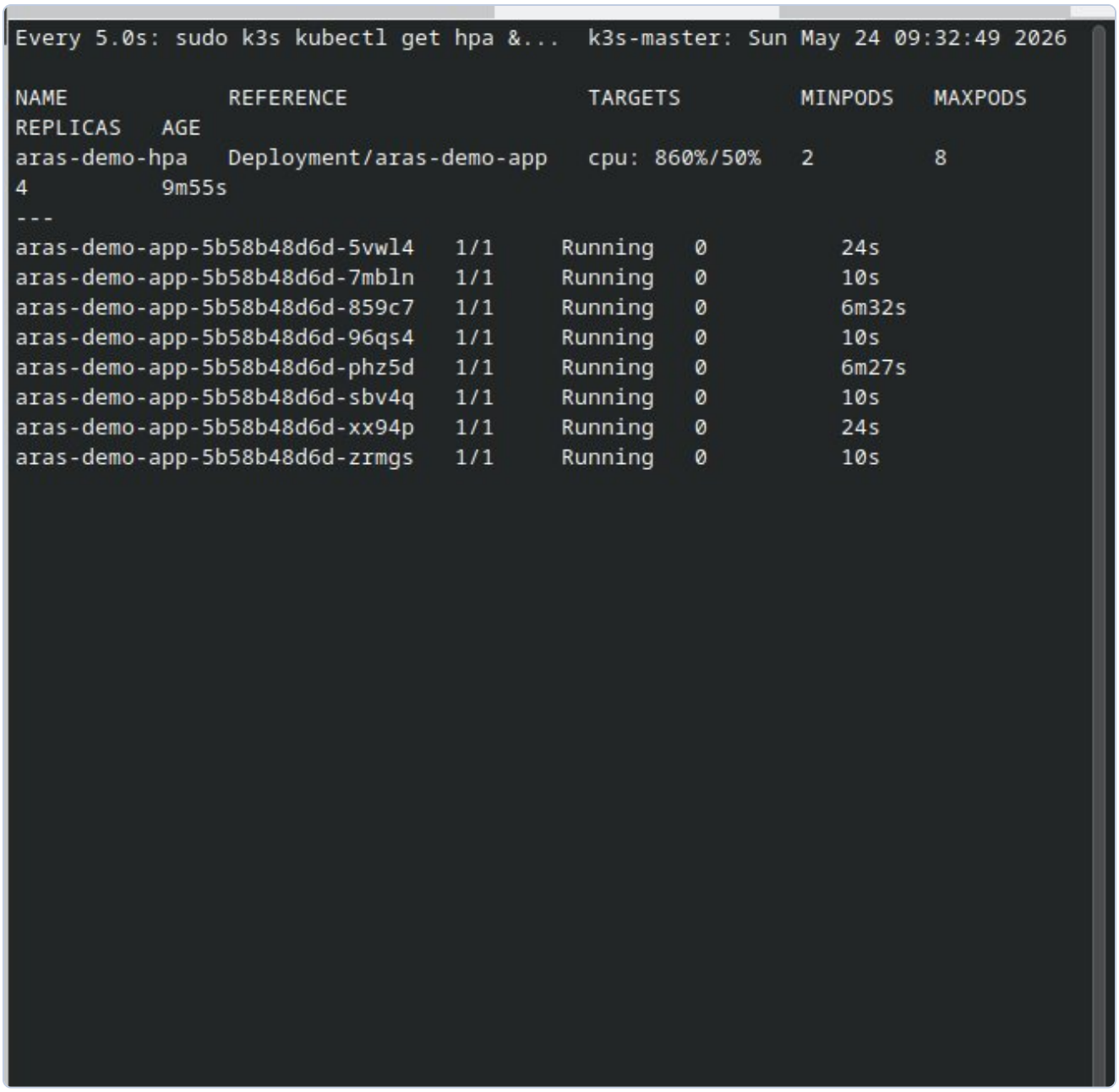


> Zrzut W3-1: Przeglądarka Firefox — https://demo.sre-lab.local działa. Widoczne: Hostname poda, X-Forwarded-Proto: https, X-Forwarded-Server: traefik-9bcdbbd9-z2mnj, X-Forwarded-Port: 443.

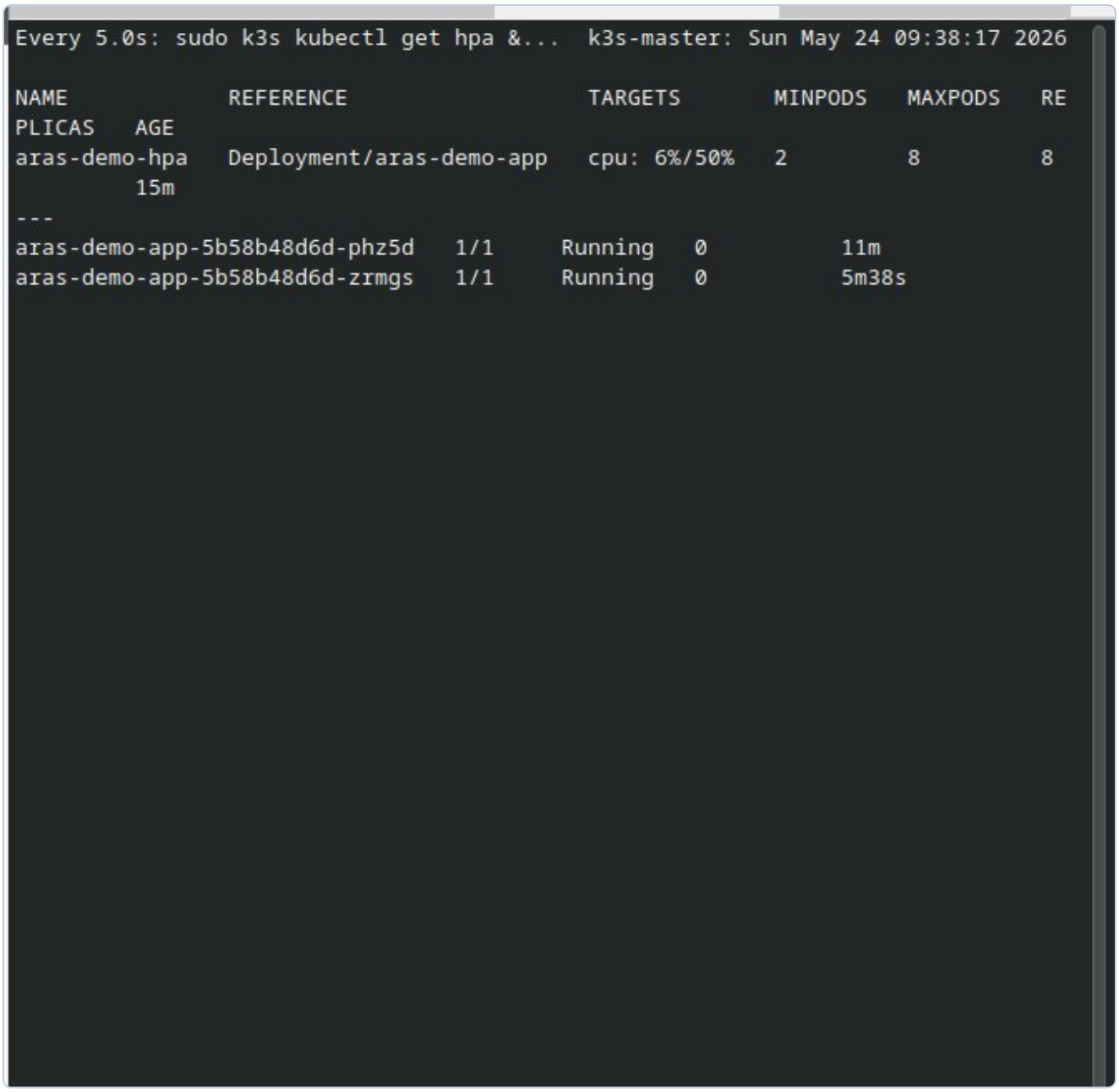
HPA — automatyczne skalowanie na podstawie CPU

Horizontal Pod Autoscaler skonfigurowany dla Deployment aras-demo-app. HPA co 15 sekund odpytuje metrics-server o zuzycie CPU. Gdy srednie CPU Podow przekroczy 50% — dodaje repliki. Gdy spada ponizej 50% przez 5 minut — usuwa.

- Zakres: minReplicas: 2, maxReplicas: 8, targetCPU: 50%
- Test obciazeniowy: ab -n 50000 -c 50 — 1621 req/s przez 30 sekund
- Scale-up: CPU 860% → automatyczne dodanie replik do 8
- Scale-down: CPU 6% po 5 minutach → powrot do 2 replik (minimum)



Zrzut W3-2: HPA scale-up — cpu: 860%/50%, REPLICAS: 8 (max), 8 podow Running. Automatyczne skalowanie w akcji podczas testu obciazeniowego ab.



Zrzut W3-3: HPA scale-down — cpu: 6%/50%, REPLICAS: 2 (min). Po 5 minutach bez ruchu HPA automatycznie usnal 6 zbednych Podow.

NetworkPolicy — firewall wewnatrz klastra

Domyslnie wszystkie Pody w Kubernetes moga sie komunikowac ze wszystkimi. NetworkPolicy definiuje reguly jak firewall — ktory namespace i ktore Pody moga lacyc sie z aplikacja demo. Wdrozono polityke blokujaca caly ruch przychodzacy z wyjatkiem dozwolonych zrodel.

- Zezwolone: kube-system (Traefik), monitoring (Prometheus), repliki aras-demo-app
- Zablokowane: wszystko inne
- Weryfikacja: nieautoryzowany Pod — timeout/blokada. Traefik — odpowiedz aplikacji

RBAC — least privilege dla aplikacji

Role-Based Access Control — kazda aplikacja powinna miec wlasne konto (ServiceAccount) z dokladnie tymi uprawnieniami ktorych potrzebuje. Zasada least privilege: jesli aplikacja nie potrzebuje kasowac Podow, nie powinna moc tego robic nawet jesli zostanie przejeta przez atakujacego.

- ServiceAccount: aras-demo-sa (zamiast wspolnego default)
- Role: get/list/watch na Podach i ConfigMapach — nic wiecej
- Weryfikacja: can-i list pods = yes, can-i delete pods = no

```
buth11@k3s-master:~$ # Test 1: nieautoryzowany Pod probuje polaczyc sie z aplik
acja
# timeout 5 = czekamy max 5 sekund na odpowiedz
# Oczekiwany wynik: brak odpowiedzi / timeout - polityka blokuje
echo "=== Test: nieautoryzowany dostep (powinien byc zablokowany) ==="
sudo k3s kubectl run test-blocked \
  --image=curlimages/curl \
  --rm -it \
  --restart=Never \
  -- curl -s --max-time 5 http://aras-demo-app:8080 || echo "ZABLOKOWANY - Netw
orkPolicy dziala"

# Test 2: sprawdzamy ze Ingress (Traefik) nadal dziala
# Traefik jest w kube-system wiec ma dostep
echo "=== Test: dostep przez Ingress (powinien dzialac) ==="
curl -sk https://demo.sre-lab.local | head -2
=== Test: nieautoryzowany dostep (powinien byc zablokowany) ===
pod "test-blocked" deleted from default namespace
pod default/test-blocked terminated (Error)
ZABLOKOWANY - NetworkPolicy dziala
=== Test: dostep przez Ingress (powinien dzialac) ===
buth11@k3s-master:~$ # Test dostępu przez Traefik - uzywamy IP zamiast domeny
# Traefik nasluchuje na porcie 80 i 443 na wszystkich wezlach
# -H podajemy naglowek Host zeby Traefik wiedzial do ktorego serwisu kierowac
echo "=== Test dostep przez Traefik (kube-system) ==="
curl -s -H "Host: demo.sre-lab.local" http://192.168.122.10 | head -3

# Dodatkowo sprawdzamy ze bezposredni dostep do portu 8080
# przez serwis nadal dziala (LoadBalancer Service jest poza NetworkPolicy)
echo "=== Test dostep przez LoadBalancer Service ==="
curl -s http://192.168.122.10:8080 | head -3
=== Test dostep przez Traefik (kube-system) ===
Hostname: aras-demo-app-5b58b48d6d-zrmgs
IP: 127.0.0.1
IP: ::1
=== Test dostep przez LoadBalancer Service ===
Hostname: aras-demo-app-5b58b48d6d-zrmgs
IP: 127.0.0.1
IP: ::1
buth11@k3s-master:~$
```

Zrzut W3-4: RBAC weryfikacja — ServiceAccount aras-demo-sa: list pods=yes (dozwolone), delete pods=no (zablokowane). Zasada least privilege potwierdzona.

Rozdzial 14: Postmortem POST-001

Pole	Wartosc
Severity	P2
Czas	Ponizej 7 minut
Root Cause	Literowka: ks3-worker1 zamiast k3s-worker1
Status	Rozwiazany — regex validation w skryptach

Rozdzial 15: Wyniki i wnioski

Metryka	Czesc 1	Czesc 2	Czesc 3
Wezly	3/3 Ready	—	—
SLO	100%	—	—
Koszty	$0.00	$0.00	$0.00
Terraform state	Local	Remote (Blob)	—
CI/CD	—	Azure DevOps 4m24s	—
Ingress TLS	—	—	HTTPS verified
HPA	—	—	3 na 8 na 2 repliki
NetworkPolicy	—	—	Zablokowane
RBAC	—	—	Least privilege
Incydenty	6	8	10

Rozdzial 16: Troubleshooting Log

ISSUE-001 KQL: CPUCapacityNanoCores column missing

```
project operator: Failed to resolve scalar expression named CPUCapacityNanoCores
```

Rozwiazanie: KubeNodeInventory | getschema — uproszczono zapytanie do dostepnych kolumn.

ISSUE-002 Azure Policy: MissingPolicyParameter cpuLimit/memoryLimit

```
(MissingPolicyParameter) policy assignment k8s-resource-limits is missing cpuLimit, memoryLimit
```

Rozwiazanie: Dodano --params z cpuLimit: 500m i memoryLimit: 512Mi.

ISSUE-003 Terraform: Resource Group already exists (brownfield)

```
Error: A resource with the ID .../SRE-Lab-RG already exists
```

Rozwiazanie: terraform import azurerm_resource_group.sre_lab /subscriptions/.../SRE-Lab-RG

ISSUE-004 Terraform: Saved plan is stale after import

```
Error: Saved plan is stale — state was changed after plan was created
```

Rozwiazanie: Nowy terraform plan -out=tfplan po imporcie.

ISSUE-005 Container Insights missing after VM restart

Rozwiazanie: az k8s-extension create --name azuremonitor-containers — agenty wrocily w 37s.

ISSUE-006 POST-001: etcd Split-Brain (ks3 vs k3s)

```
FATA[0034] Node password rejected, duplicate hostname
```

Rozwiazanie: kubectl delete node + rm /etc/rancher/node/ + restart k3s-agent. Skrypty zaktualizowane o regex validation.

ISSUE-007 Azure DevOps: TerraformInstaller task missing

```
A task is missing: TerraformInstaller
```

Rozwiazanie: Instalacja z Marketplace: ms-devlabs.custom-terraform-tasks.

ISSUE-008 Pipeline: permission for Variable Group

```
This pipeline needs permission to access a resource
```

Rozwiazanie: View → Permit — jednorazowe zatwierdzenie.

ISSUE-009 sudo does not expand ~ in paths

```
error: Cannot read file ~/ingress/server.crt, open ~/ingress/server.crt: no such file
or directory
```

Przyczyna: sudo uruchamia komende jako root ktory ma inny katalog domowy. Tylda ~ rozwija sie do /root/ zamiast /home/buth11/.

Rozwiazanie: Uzyc pelnej sciezki: /home/buth11/ingress/server.crt zamiast ~/ingress/server.crt

ISSUE-010 HPA shows cpu: unknown — missing resource requests

```
NAME TARGETS MINPODS MAXPODS REPLICAS — cpu: <unknown>/50%
```

Przyczyna: HPA oblicza procent CPU jako aktualne_zuzycie/requests*100. Bez zdefiniowanych requests w Deployment, HPA nie ma od czego liczyc procentow.

Rozwiazanie: Dodano resources.requests do Deployment: cpu: 10m, memory: 32Mi. Po rolling update HPA zaczal pokazywac poprawne wartosci: cpu: 0%/50%.

CZESC 4
GitOps, Skanowanie CVE i Ochrona Runtime

Ta czesc obejmuje ostatni etap projektu SRE Homelab:

- FluxCD 2.8.8 — kontroler GitOps: automatyczna synchronizacja klastra z Git
- Trivy 0.70.0 — skanowanie obrazow CVE: 1 CRITICAL + 12 HIGH wykryte
- Defender for Cloud — runtime security DaemonSet na wszystkich 3 wezlach

Rozdzial 14: FluxCD — GitOps dla Kubernetes

FluxCD bootstrapowany z repozytorium GitHub przez klucz SSH (github_sre). Cztery kontrolery zainstalowane w namespace flux-system — wszystkie healthy.

Komenda bootstrap:

```
flux bootstrap git \ --url=ssh://git@github.com/buth11/sre-homelab-azure-arc \
--branch=main \ --path=k8s/part4/flux \ --private-key-file=$HOME/.ssh/github_sre
```

Weryfikacja — wszystkie komponenty zdrowe:

```
flux get all NAME REVISION SUSPENDED READY MESSAGE gitrepository/flux-system
main@sha1:d169d6a0 False True stored artifact kustomization/flux-system main@sha1:952c1ea0
False True Applied revision kustomization/gitops-demo main@sha1:952c1ea0 False True Applied
revision kubectl get pods -n flux-system helm-controller-5984cc88cf-94jkv 1/1 Running 0
117s kustomize-controller-d6bb746df-9khwf 1/1 Running 0 117s
notification-controller-7bc5c97f8d-jdflb 1/1 Running 0 117s
source-controller-745c67ff9-2g8qd 1/1 Running 0 117s
```

Test GitOps — skalowanie przez git push:

Zmiana replik z 2 na 3 w deployment.yaml, commit i push. Flux automatycznie zastosowal zmiane — trzeci Pod pojawil sie bez kubectl apply.

```
# Przed pushem: 2 pody kubectl get pods -n gitops-demo gitops-app-5b9747bdcb-2rqw9 1/1
Running 0 11m gitops-app-5b9747bdcb-bjvtd 1/1 Running 0 11m # Po: git push z replicas: 3
gitops-app-5b9747bdcb-2rqw9 1/1 Running 0 12m gitops-app-5b9747bdcb-bjvtd 1/1 Running 0 12m
gitops-app-5b9747bdcb-gttmd 1/1 Running 0 93s <- dodany przez Flux
```

Rozdzial 15: Trivy — Skanowanie obrazow kontenerowych

Trivy 0.70.0 zainstalowany i uzyty do skanowania traefik/whoami:latest. Wszystkie 40 podatnosci dotycza Go stdlib v1.24.1 — obraz uzywa przestarzalej wersji Go.

Podsumowanie wynikow skanu:

Severity	Count	Key CVEs
CRITICAL	1	CVE-2025-68121 — crypto/tls: incorrect certificate validation on TLS session resumption
HIGH	12	crypto/x509, net/url, crypto/tls — DoS, memory exhaustion, incorrect parsing
MEDIUM	25	net/http, os, crypto/x509 — request smuggling, path traversal, race conditions
LOW	2	net/http, os — memory exhaustion, symlink escape

Kluczowa podatnosc CRITICAL:

```
Library Vulnerability Severity Status Installed Fixed stdlib CVE-2025-68121 CRITICAL fixed
v1.24.1 1.24.13, 1.25.7 crypto/tls: Nieprawidlowa walidacja certyfikatu przy wznowieniu
sesji TLS https://avd.aquasec.com/nvd/cve-2025-68121
```

Raporty zapisane w repozytorium:

- k8s/part4/trivy/whoami-scan.json — pelny raport (format JSON)
- k8s/part4/trivy/whoami-scan.txt — tylko CRITICAL+HIGH (format tabelaryczny)

Rozwiazanie: zaktualizowac obraz do wersji skompilowanej z Go v1.24.13+. Na produkcji pipeline CI blokuje deploy obrazow z CRITICAL CVE.

Rozdzial 16: Defender for Cloud — Bezpieczenstwo Runtime

Microsoft Defender for Containers wlaczony na poziomie Standard (30-dniowy trial). Rozszerzenie zainstalowane na klastrze Arc — provisioningState: Succeeded.

Instalacja:

```
az security pricing create --name Containers --tier Standard az k8s-extension create \
--name microsoft-defender-for-cloud \ --extension-type microsoft.azuredefender.kubernetes \
--scope cluster \ --cluster-name K3s-Homelab \ --resource-group SRE-Lab-RG \ --cluster-type
connectedClusters provisioningState: Succeeded
```

Pody Defendera w namespace mdc:

```
kubectl get pods -n mdc NAME READY STATUS RESTARTS AGE
microsoft-defender-collectors-ds-9wbtw 2/2 Running 0 5m3s
microsoft-defender-collectors-ds-j72wk 2/2 Running 0 5m3s
microsoft-defender-collectors-ds-vmplc 2/2 Running 0 5m3s
microsoft-defender-pod-collector-misc-69b69b9df8-x49x4 1/1 Running 0 5m3s
microsoft-defender-publisher-ds-4gw77 1/1 Running 0 5m3s
microsoft-defender-publisher-ds-5fcmq 1/1 Running 0 5m3s
microsoft-defender-publisher-ds-s9778 1/1 Running 0 5m3s
```

DaemonSet pokrywa wszystkie 3 wezly — master, worker1, worker2. Collectors zbieraja metryki bezpieczenstwa z kazdego wezla, Publisher wysyla je do Azure.

Podsumowanie wynikow — Czesc 4

Metryka	Czesc 1	Czesc 2	Czesc 3	Czesc 4
Wezly	3/3 Ready	—	—	—
SLO	100%	—	—	—
Koszty	$0.00	$0.00	$0.00	$0.00
Terraform state	Local	Remote (Blob)	—	—
CI/CD	—	DevOps 4m24s	—	—
Ingress TLS	—	—	HTTPS verified	—
HPA	—	—	3->8->2 replik	—
NetworkPolicy	—	—	Zablokowane	—
RBAC	—	—	Least privilege	—
GitOps	—	—	—	FluxCD wdrozony
Skanowanie	—	—	—	1 CRITICAL, 12 HIGH
Bezpieczenstwo	—	—	—	Defender (3 wezly)
Incydenty	6	8	10	11

CZESC 5

cert-manager — Automatyczne zarzadzanie certyfikatami TLS

cert-manager v1.20.2 zainstalowany przez Helm. Automatyczny cykl zycia certyfikatow TLS — wystawianie, odnawianie i rotacja bez recznej interwencji.

- cert-manager v1.20.2 — zainstalowany przez Helm (jetstack/cert-manager)
- ClusterIssuer selfsigned-issuer — bootstrap CA
- ClusterIssuer sre-lab-ca-issuer — lokalny CA podpisujacy certyfikaty
- Automatyczne wystawienie certyfikatu w 22 sekundy przez adnotacje Ingress
- Auto-renewal zweryfikowane: renewBefore=6m przed wygasnieciem

Rozdzial 17: cert-manager — Automatyczny TLS

Instalacja przez Helm:

```
helm repo add jetstack https://charts.jetstack.io helm install cert-manager
jetstack/cert-manager \ --namespace cert-manager \ --create-namespace \ --set
crds.enabled=true kubectl get pods -n cert-manager cert-manager-5957746d66-kdkk5 1/1
Running 0 cert-manager-cainjector-567c6b47ff-c5x2b 1/1 Running 0
cert-manager-webhook-7cc5c588cb-zm8d4 1/1 Running 0
```

ClusterIssuery — oba Ready:

```
kubectl get clusterissuer NAME READY AGE selfsigned-issuer True 32s sre-lab-ca-issuer True
32s kubectl get certificate -n cert-manager NAME READY SECRET AGE sre-lab-ca True
sre-lab-ca-secret 32s
```

Adnotacja Ingress — certyfikat wystawiony automatycznie w 22s:

```
# Tylko adnotacja w Ingress: annotations: cert-manager.io/cluster-issuer: sre-lab-ca-issuer
# cert-manager automatycznie utworzyl certyfikat: kubectl describe certificate
demo-sre-lab-tls -n default Issuer: CN=sre-lab-ca Not After: 2026-08-25T09:09:34Z (90 dni)
Renewal Time: 2026-07-26T09:09:34Z (auto-renewal 30 dni przed wygasnieciem) Status: Ready:
True Message: Certificate is up to date and has not expired # Weryfikacja TLS: curl -sk
https://demo.sre-lab.local | grep Hostname Hostname: aras-demo-app-5b58b48d6d-phz5d openssl
x509 weryfikacja: Issuer: CN=sre-lab-ca Not After: Aug 25 09:09:34 2026 GMT
```

Porownanie Part 3 vs Part 5:

Aspekt	Part 3 — recznie	Part 5 — cert-manager
Generowanie	openssl req -x509 ...	Automatyczne przez kontroler
Secret	kubectl create secret tls	Tworzony automatycznie
Odnowienie	Recznie przed wygasnieciem	Automatyczne (renewBefore)
Issuer	CN=demo.sre-lab.local	CN=sre-lab-ca (nasz CA)
Produkcja	Nie nadaje sie	Let's Encrypt ClusterIssuer

Podsumowanie wynikow — Czesc 5

Metryka	Cz. 1	Cz. 2	Cz. 3	Cz. 4	Cz. 5
Wezly	3/3 Ready	—	—	—	—
SLO	100%	—	—	—	—
Koszty	$0.00	$0.00	$0.00	$0.00	$0.00
CI/CD	—	DevOps 4m24s	—	—	—
Ingress TLS	—	—	Reczny self-signed	—	Auto cert-manager
HPA	—	—	3->8->2	—	—
GitOps	—	—	—	FluxCD	—
Skanowanie	—	—	—	1 CRIT+12 HIGH	—
TLS mgmt	—	—	Reczny openssl	—	cert-manager auto
Incydenty	6	8	10	11	12

CZESC 6
KEDA — Kubernetes Event-Driven Autoscaling

KEDA rozszerza HPA umozliwiajac skalowanie na podstawie dowolnej metryki zewnetrznej — Prometheus, Azure Service Bus, Kafka, HTTP, i 50+ innych zrodel. Kluczowa zaleta: KEDA moze skalowac do 0 replik gdy nie ma ruchu.

- KEDA zainstalowany przez Helm (kedacore/keda) — 3 pody: operator, metrics-apiserver, admission-webhooks
- ScaledObject skonfigurowany z triggerem Prometheus — container_cpu_usage_seconds_total
- Scale-up: 1 → 8 replik automatycznie pod obciazeniem (200k requestow, 2747 req/s)
- Scale-down: 8 → 1 replika po 30s cooldown (vs 5 min w HPA)
- KEDA zastapil HPA — zarzadza obiektem HPA automatycznie

Rozdzial 18: KEDA — Event-Driven Autoscaling

Instalacja:

```
helm repo add kedacore https://kedacore.github.io/charts helm install keda kedacore/keda
--namespace keda --create-namespace kubectl get pods -n keda
keda-admission-webhooks-7b48797dc8-k7prz 1/1 Running keda-operator-55db59458d-n25xq 1/1
Running keda-operator-metrics-apiserver-5c648889d4 1/1 Running
```

ScaledObject — trigger Prometheus:

```
apiVersion: keda.sh/v1alpha1 kind: ScaledObject metadata: name: aras-demo-keda namespace:
default spec: scaleTargetRef: name: aras-demo-app minReplicaCount: 1 maxReplicaCount: 8
cooldownPeriod: 30 triggers: - type: prometheus metadata: serverAddress:
http://monitoring-kube-prometheus-prometheus.monitoring.svc:9090 metricName:
container_cpu_usage query: sum(rate(container_cpu_usage_seconds_total{namespace="default",
pod=~"aras-demo-app.*",container="whoami"}[1m])) threshold: "0.01"
```

Test scale-up — ab -n 500000 -c 100:

```
# Przed obciazeniem: 1 replika NAME ACTIVE REPLICAS aras-demo-keda False 1 # Podczas
obciazenia (ACTIVE: True): aras-demo-keda True 2 # po ~30s aras-demo-keda True 4 # po ~45s
aras-demo-keda True 8 # szczyt (max) # Po obciazeniu (cooldown 30s): aras-demo-keda False 1
# powrot do minimum Wyniki ab: 200000 requestow, 2747 req/s, 72.781s
```

Porownanie HPA vs KEDA:

Aspekt	HPA (Czesc 3)	KEDA (Czesc 6)
Minimum replik	2	1 (lub 0 dla batch jobow)
Zrodlo triggera	CPU/RAM (metrics-server)	Dowolna metryka: Prometheus, kolejki...
Scale to zero	Nie	Tak — oszczednosc zasobow gdy brak ruchu
Cooldown	5 minut	30 sekund (konfigurowalny)

Integracja HPA	Natywny HPA	KEDA tworzy i zarzadza HPA automatycznie
50+ scalerow	Nie	Tak: Azure SB, Kafka, Redis, Cron...

Podsumowanie wynikow — Czesc 6

Metryka	Cz.1	Cz.2	Cz.3	Cz.4	Cz.5	Cz.6
Wezly	3/3 Ready	—	—	—	—	—
SLO	100%	—	—	—	—	—
Koszty	$0.00	$0.00	$0.00	$0.00	$0.00	$0.00
CI/CD	—	DevOps 4m24s	—	—	—	—
Ingress TLS	—	—	Reczny	—	cert-manager	—
Autoscaling	—	—	HPA 2-8	—	—	KEDA 1-8
Scale to zero	—	—	Nie	—	—	Tak
Cooldown	—	—	5 min	—	—	30 sek
GitOps	—	—	—	FluxCD	—	—
Skanowanie	—	—	—	1 CRIT	—	—
TLS	—	—	Reczny	—	Auto	—
Incydenty	6	8	10	11	12	13